ReUse!

Fase 3 - Desenvolvimento mobile

Consumo de APIs, autenticação e sessões, e caching local em React Native.

Integrante	RM
Emanuel Secco Eidt	567455
Laura Franco Pigosso	562157
Mariana Rodrigues Trindade da Silva	567137
Marcos Nunes	567631

Repositório com todo o desenvolvimento React Native: https://github.com/mmarianatrindade/Re-Use

O ReUse conecta pessoas que desejam vender, trocar ou doar itens usados. Nesta fase, o aplicativo foi evoluído para buscar dados de serviços externos ao app, manter sessões autenticadas e reutilizar informações armazenadas no aparelho.

Stack: Expo SDK 53, React Native 0.79, React 19, TypeScript, React Navigation 7 e formulários com React Hook Form e Zod. O back-end próprio utiliza Cloudflare Workers e D1. O código do app está em src/ e o da API em api/.

1. Consumo de APIs

API	Aplicação no projeto
API ReUse - REST/JSON	Contas, sessão, perfil, anúncios, fotos, denúncias e estatísticas. É um serviço próprio, acessado pelo aplicativo via HTTP.
ViaCEP - pública	Consulta o CEP e preenche rua, bairro, cidade e UF no cadastro e na edição do perfil.
AwesomeAPI CEP - pública	Consulta coordenadas aproximadas do CEP para apoiar os filtros por localização.
Google Places - opcional	Sugestões e detalhes de endereço, via proxy na API ReUse. Depende da chave configurada no servidor.
Nominatim e BrasilAPI	Fontes de reserva usadas pela API ReUse na geocodificação do CEP, conforme a disponibilidade dos serviços.

Base configurada para a API ReUse: https://reuse-api.r3vmarcos.workers.dev. A variável EXPO_PUBLIC_API_URL permite selecionar essa API ou uma instância local. O comportamento documentado corresponde ao código desta entrega; atualizar o GitHub não publica automaticamente alterações no serviço hospedado.

1.1 Telas que consomem APIs

Tela	Dados e operações
Início	GET /anuncios: carrosséis de venda, troca e doação. GET /estatisticas: indicadores calculados a partir dos registros da plataforma.
Catálogo (Anúncios)	GET /anuncios com busca, modalidade, categoria, preço, entrega e localização. Paginação e atualização da lista.
Detalhe do anúncio	GET /anuncios/:id; POST /anuncios/:id/visualizacao; POST /anuncios/:id/denuncia.
Publicar e Editar anúncio	POST /uploads para fotos; POST /anuncios para publicar; PUT /anuncios/:id para editar.
Negociações e Gerenciar anúncio	GET /meus-anuncios e /meus-anuncios/:id; mudança de status e exclusão dos anúncios da própria conta.
Destacar anúncio	Criação e cancelamento de pedido de destaque. O pedido fica aguardando pagamento; a integração de pagamento pertence a uma etapa futura.
Meu perfil e Editar perfil	GET e PATCH /me; upload da foto do perfil.
Perfil público	GET /usuarios/:id, com dados públicos do anunciante.
Cadastro e Editar perfil - endereço	ViaCEP, AwesomeAPI e, quando habilitado, o proxy de Google Places em /enderecos/*.
Entrar, Cadastro e fluxos de conta	POST /auth/* para criar conta, entrar, renovar, sair e tratar links de ativação ou recuperação.

1.2 Exemplos de uso e tratamento de falhas

No cadastro, ao completar um CEP, o app consulta o ViaCEP e preenche os campos do endereço. Se a consulta falhar ou o CEP não for encontrado, a pessoa pode preencher os campos manualmente. ViaCEP e AwesomeAPI têm limite de espera de 5 segundos por consulta; uma falha na obtenção de coordenadas não impede, por si só, o envio do cadastro.

No catálogo, a busca e os filtros compõem os parâmetros de GET /anuncios. A resposta contém os cartões e a indicação da próxima página. O app utiliza esses dados para atualizar a lista e abrir o detalhe de cada item.

As chamadas à API ReUse passam por src/services/api.ts. As operações autenticadas enviam Authorization: Bearer. O cliente trata ausência de conexão, tempo esgotado, JSON inválido, 401 e erros de validação 422 com mensagens por campo. Cada tentativa tem limite de 15 segundos, ou 60 segundos para upload. GET pode repetir uma vez em falhas de rede ou respostas 502, 503 e 504; gravações não são repetidas automaticamente.

2. Autenticação e sessões

Tela ou fluxo	Aplicação da autenticação
Entrar	Login com e-mail e senha. O último e-mail usado no login ou cadastro pode ser preenchido na próxima abertura.
Cadastro	Criação da conta e início da sessão. Há validação de CPF, maioridade e aceite dos termos. A foto selecionada é enviada após a criação da conta.
Anunciar e Negociações	AuthGate solicita login ou cadastro quando não há sessão. Negociações concentra a gestão dos próprios anúncios nesta fase.
Meu perfil	Área protegida, com dados privados da própria conta.
Editar perfil, Gerenciar, Editar e Destacar anúncio	Requisições com token. A API verifica a sessão e, nos anúncios, a propriedade do recurso.
Denunciar anúncio	Exige sessão; sem login, o app direciona para Entrar. Salvar um anúncio é uma ação local e não exige login.
Ativação e recuperação	Telas e rotas de link implementadas. O envio real de e-mail depende da configuração do ambiente, conforme explicado abaixo.

2.1 Persistência, renovação e encerramento

O servidor gera tokens opacos de 256 bits e armazena apenas seu hash SHA-256. A sessão tem validade de 7 dias. O app verifica a necessidade de renovação na abertura, ao retornar ao primeiro plano e a cada hora, renovando quando faltam menos de 2 dias para expirar.

No Android e iOS, token, validade e identificador da conta são armazenados com expo-secure-store. Na web, a sessão usa AsyncStorage do navegador, acessível a JavaScript da origem; essa alternativa não possui a mesma proteção do armazenamento seguro nativo.

Ao abrir o app, a sessão salva é verificada e o perfil é consultado em GET /me. Sem conexão, uma sessão ainda válida pode permanecer ativa, embora o perfil possa ficar indisponível até uma nova consulta. A API continua responsável por autorizar as operações remotas.

Ao sair ou expirar, o estado local é encerrado e os dados associados à conta são limpos. Respostas de operações anteriores são conferidas para não restaurar uma sessão encerrada nem gravar dados na conta seguinte. Um 401 de uma credencial substituída não encerra a sessão nova. A renovação compartilha uma única operação no app e a rotação no servidor usa um lote transacional.

2.2 Configuração de e-mail nesta entrega

A configuração incluída mantém a confirmação de e-mail desativada e não habilita o provedor de envio. Em desenvolvimento, o fluxo pode ser demonstrado com o linkDev, exibido pelo app. Em produção sem provedor, a recuperação retorna indisponibilidade; não informa que enviou um e-mail. O recebimento real de e-mails não integra as verificações realizadas. Quando uma redefinição válida é concluída, a API revoga as sessões da conta.

3. Caching local

O aplicativo utiliza AsyncStorage para dados de navegação e conteúdo reutilizável. Nas telas com cache de respostas, a cópia válida pode ser mostrada primeiro e a rede fornece a atualização. Se a atualização falhar, a tela pode manter os dados existentes com um aviso.

Item armazenado	Telas	Validade ou remoção
Primeira página dos carrosséis de venda, troca e doação	Início	6 horas
Primeira página por combinação de busca e filtros	Catálogo	6 horas
Detalhe do anúncio consultado	Detalhe do anúncio	24 horas; removido quando a API responde 404
Indicadores da plataforma	Início	24 horas
Endereço retornado pelo ViaCEP	Cadastro e Editar perfil	30 dias
Lista Meus anúncios	Negociações	Até atualização, saída ou expiração da conta
Rascunho com campos, fotos e etapa	Publicar	Salvo após 600 ms sem alterações ou ao avançar; removido ao publicar, descartar, sair ou expirar
Cartões dos anúncios salvos	Salvos e Detalhe	Até remoção pela pessoa; permanecem no aparelho após sair
Último e-mail usado	Entrar	Até novo login ou cadastro; permanece após sair

3.1 Regras e privacidade

O módulo src/services/cache.ts usa chaves versionadas (v2), prazo de validade por tipo e limite de 40 respostas. Esse limite se aplica ao módulo de respostas, não à lista de favoritos ou aos rascunhos. As gravações são serializadas e as versões antigas do cache são removidas na inicialização.

Meus anúncios e rascunhos usam chaves vinculadas ao identificador da conta. As respostas de listas e detalhes que dependem da sessão também são separadas por contexto. A geração da conta é verificada antes de gravar resultados assíncronos, evitando que uma resposta antiga repovoe os dados após a saída.

O cache de respostas não armazena senha, CPF ou telefone do cadastro. O rascunho contém os campos do anúncio, não os dados cadastrais privados. A credencial de sessão é armazenada separadamente, conforme as diferenças entre mobile e web descritas na seção 2.1. A posição do GPS é reduzida na chave da consulta, sem persistir ali sua precisão original.

As imagens remotas utilizam expo-image com cache em memória e disco. O app apresenta aviso de falta de conexão e, no detalhe, informa a idade da cópia utilizada quando a rede falha.

4. Execução e verificações

O repositório inclui package-lock.json para registrar as versões instaladas. A preparação local utiliza Node.js 24 e npm. Os testes usam o executor do Node e SQLite em memória, além de simulações de rede, armazenamento e funções de React.

Verificação	Resultado nesta revisão
npm run typecheck	Checagem TypeScript do app e da API aprovada.
npm test	16 testes de regressão aprovados.
npm run export:web	Exportação web do aplicativo gerada com sucesso.
Testes em aparelhos Android/iOS	Não executados nesta revisão.
Recebimento real de e-mail e publicação em Cloudflare	Não realizados nesta revisão.

Os testes cobrem tratamento de 401, erros 422, JSON inválido, timeout, repetição de GET, limite e validade do cache, separação de contas, respostas após logout, renovação concorrente, falha de CEP, indisponibilidade do envio de e-mail, limitação de requisições e validação de uploads. São verificações automatizadas de lógica; não equivalem à execução completa dos fluxos em um aparelho.

4.1 Como executar a demonstração local

Instalar dependências: npm ci.

Preparar a configuração local: npm run demo:configurar. O comando cria .env e api/.dev.vars apenas se ainda não existirem e gera uma chave de desenvolvimento para o CPF. Os dois arquivos são ignorados pelo Git.

Aplicar todas as migrações, inclusive a tabela de limites: npm run api:migrar:local.

Em um terminal, executar npm run api:dev. Em outro, executar npm start e selecionar a plataforma. No celular físico, ajustar EXPO_PUBLIC_API_URL no .env para o IP do computador acessível pela mesma rede; localhost no celular não aponta para o computador.

4.2 Limites do escopo atual

Propostas, chat, pagamentos e avaliações completas pertencem a etapas futuras. O detalhe permite salvar e compartilhar anúncios; Negociações permite gerenciar as próprias publicações. A busca por proximidade considera até 500 candidatos, devendo ser revista para um catálogo maior.

A API limita a frequência de operações como login, cadastro, consultas de endereço e uploads. O envio de imagens confere o tipo e a assinatura do arquivo, limita o corpo a 1,5 MB e estabelece cota de até 100 arquivos e 50 MB por conta; essa validação não equivale a uma análise antivírus ou decodificação completa da imagem.

Revisão da entrega: 2 de novembro de 2026.